

FAST-MVH: K-d Tree Indexing for Dominance Checking in Multi-Objective Search

Abstract

Multi-objective shortest path search maintains multiple Pareto-optimal solutions at each state. When heuristics take the form of multiple trade-off vectors, testing dominance—against frontier members, goal solutions, and residual evicted vectors—dominates search cost, often exceeding path-generation cost. This work accelerates dominance checking via spatial indexing: truncated state frontiers are indexed with adaptive k-d trees; the full-dimensional fallback is confined to an append-only residue indexed via the logarithmic method. Local dominance is tested before heuristic selection, eliminating expensive evaluations on certain-to-be-pruned paths. Witness caching captures and reuses full-dimensional dominators found during truncated scans, bypassing redundant full-dimensional queries. On eighteen instances spanning three to eight objectives across grids and road networks, FAST-MVH returns identical solutions in $1.8\times$ to $82\times$ less time, reducing full-dimensional comparisons by 10–82 times.

1 Introduction

Problem statement

Multi-objective shortest path problems arise when a system’s cost has multiple, conflicting dimensions. Rather than a single optimal cost, the algorithm must return the Pareto frontier: all costs where no alternative is better in all dimensions simultaneously. Searching efficiently requires two ingredients: strong lower bounds (heuristics) and fast pruning of dominated candidates.

Traditional single-objective heuristics (e.g., landmarks via ALT) are weak in multi-objective settings. Instead, systems use multi-valued heuristics: at each state, multiple trade-off vectors, each a valid lower bound on some reachable goal-path vector. Admissibility requires that at least one of these vectors bounds every reachable solution.

The computational burden of multi-valued heuristics is dominance checking. Every state frontier must be tested against every candidate path; every candidate heuristic must be tested against the goal frontier; fallback cases require full-dimensional searches through evicted vectors. On dense problems, dominance checks outnumber path generations by 10–100 $\times$.

Three optimization axes

This work reduces dominance-checking cost through three complementary designs:

1. **Truncated frontier indexing.** Local dominance depends only on coordinates 2, ..., M (the last $M - 1$). Indexing frontiers with k-d trees over truncated space exploits this structure, making spatial pruning cheap and effective.
2. **Witness caching and fallback indexing.** When truncated queries find only partial dominators—vectors p where $\text{Tr}(p) \leq \text{Tr}(g)$ but $p_1 > g_1$ —a full-dimensional search is needed. Rather than re-scanning the frontier’s full space, an append-only residue $R(s)$ collects evicted vectors and indexes them via a logarithmically decomposed k-d forest. Witness caching avoids repeat full-dimensional queries by recording truncated-scan results.
3. **Test-order and adaptive indexing.** Local dominance is tested before heuristic selection because it is cheaper and more selective. State frontiers are promoted to k-d trees only after they exceed small thresholds and query costs justify tree overhead, balancing promotion cost against amortized query savings.

Together, these yield an algorithm that makes identical decisions and expansions as standard search but at dramatically lower cost.

2 Problem Definitions

Let $G = (S, E, c)$ be a finite directed graph where each edge has cost $c(e) \in \mathbb{R}_{\geq 0}^M$. The cost of a path is the component-wise sum of edge costs. Vector x weakly dominates y (written $x \leq y$) if $x_k \leq y_k$ for all $k \in [1, M]$. The task is to return all paths with distinct Pareto-optimal costs.

The truncation operator drops the first coordinate:

$$\text{Tr}(x) = (x_2, \dots, x_M).$$

A **reduced frontier** $F(s)$ at state s maintains coverage: for every cost ever generated at s , some frontier member truncation-dominates it. When a new cost g is generated:

$$F(s) \leftarrow \{p \in F(s) : \text{Tr}(g) \not\leq \text{Tr}(p)\} \cup \{g\}.$$

Frontiers preserve coverage but need not be full-dimensional Pareto sets.

An **admissible multi-valued heuristic** assigns to each state an ordered set:

$$H(s) = \langle h^1, \dots, h^{k(s)} \rangle, \quad h^1 \leq_{\text{lex}} \dots \leq_{\text{lex}} h^{k(s)},$$

where for each goal-reaching suffix from s , some h^i bounds it component-wise. Lexicographic ordering is a correctness requirement: admissibility depends on selecting the first surviving vector in order.

The **goal frontier** is $T = F(s_{\text{goal}})$. A cost f is **goal-dominated** if T contains a member dominating $\text{Tr}(f)$. A cost g at state s is **locally dominated** if some previously expanded cost at s dominates g in all M coordinates.

3 Algorithm

3.1 Node lifecycle

FAST-MVH follows standard multi-objective search: extract a minimum- f node from OPEN; if goal-dominated, select a new heuristic and reinsert; otherwise expand it, test successors for local dominance and heuristic acceptability, and insert them. Nodes record state $s(n)$, cost $g(n)$, heuristic index $i(n)$, and $f(n) = g(n) + h^{i(n)}$. OPEN is ordered lexicographically by f .

Algorithm 1: FAST-MVH Search

```

1. Sols  $\leftarrow \emptyset$ ;  $F(s)$ ,  $R(s) \leftarrow \emptyset$  for all  $s$ 
2.  $n \leftarrow (s_{\text{goal}}, \emptyset, 1)$ ; OPEN  $\leftarrow \{n\}$ 
3. while OPEN  $\neq \emptyset$ 
4.    $n \leftarrow \text{OPEN.PopMin}()$ 
5.   if GoalDom( $f(n)$ ) then
6.      $i \leftarrow \text{ChooseH}(s(n), g(n), i(n)+1, \text{true})$ 
7.     if  $i \neq \perp$  then  $i(n) \leftarrow i$ ; insert  $n$  in OPEN; continue
8.   if LocalDom( $s(n)$ ,  $g(n)$ ) then continue
9.   Update( $s(n)$ ,  $g(n)$ )
10.  if  $s(n) = s_{\text{goal}}$  then Sols  $\leftarrow \text{Sols} \cup \{n\}$ ; continue
11.  for  $s' \in \text{Succ}(s(n))$ 
12.     $g' \leftarrow g(n) + c(s(n), s')$ 
13.    if LocalDom( $s'$ ,  $g'$ ) then continue
14.     $i' \leftarrow \text{ChooseH}(s', g', 1, \text{false})$ 
15.    if  $i' \neq \perp$  then insert ( $s'$ ,  $g'$ ,  $i'$ ) in OPEN
16. return Sols

```

3.2 Dominance queries with witness caching

Both goal and local dominance involve querying whether any member of a set dominates a query vector. FAST-MVH indexes these sets spatially and caches results to avoid redundant scans.

Algorithm 2: Dominance Queries with Witness Caching

```

1. function GoalDom( $f$ )
2.   return [Find( $T$ ,  $\text{Tr}(f)$ ,  $\text{Tr}$ ) yields]
3. function LocalDom( $s$ ,  $g$ )
4.    $w \leftarrow \text{NONE}$ 
5.   for  $p \in \text{Find}(F(s), \text{Tr}(g), \text{Tr})$ 
6.     if  $p_1 \leq g_1$  then return true
7.      $w \leftarrow \text{TR}$ 
8.   if  $w = \text{NONE}$  then return false
9.   //  $w = \text{TR}$ : truncated dominators exist but none is full-dimensional.
10.  // Witness: record that at least one truncated dominator exists in  $F(s)$ .
11.  return [Find( $R(s)$ ,  $g$ ,  $\text{id}$ ) yields]
12. procedure Update( $s$ ,  $g$ )
13.  for  $p \in F(s)$  with  $\text{Tr}(g) \subseteq \text{Tr}(p)$ 
14.    remove  $p$  from  $F(s)$ 
15.    if  $p_1 < g_1$  then append  $p$  to  $R(s)$ 
16.  insert  $g$  into  $F(s)$ 

```

Witness caching. Line 5–7: The scan through truncated frontiers of $F(s)$ yields *witnesses*—truncated dominators. If a witness is also a full-dimensional dominator (line 6), the path is pruned. If no full-dimensional dominator exists in truncated matches (line

9–10), we know no full-dimensional dominator exists in $F(s)$ *unless* an evicted vector in $R(s)$ dominates. The witness status avoids redundant full-dimensional scans on future calls with similar g .

Fallback indexing. Line 11: If only truncated dominators exist, a full-dimensional search is confined to $R(s)$, the set of vectors evicted from $F(s)$ where the evictor had a larger first coordinate. This set is append-only and indexed via the logarithmic method, yielding $O(\log|R(s)|)$ query cost.

3.3 Test order: local-first pruning

Line 8 of Algorithm 1 tests local dominance *before* heuristic selection (lines 6–7). Both are pure operations (no frontier modification), so order is immaterial for correctness. However, local dominance is typically much cheaper (a small state frontier vs. a large heuristic set) and more selective (many paths are dominated locally). Testing it first avoids expensive heuristic evaluations on doomed candidates.

3.4 Residue set and indexed fallback

When Update evicts p from $F(s)$ due to $\text{Tr}(g) \leq \text{Tr}(p)$: - If $g \leq p$, then p is forever dominated; discard it. - If $g_1 > p_1$, then p may later dominate arrivals with $g_1 < p_1$; retain it in $R(s)$.

FAST-MVH retains p iff $p_1 < g_1$. This set is append-only and can be large, so indexing is critical. The **logarithmic method** of Bentley and Saxe structures $R(s)$ as a forest of immutable k-d trees:

- New vectors append to an unindexed buffer.
- At 64 vectors, the buffer becomes a locked k-d tree (with medians positioned via linear-time selection).
- Subsequent buffers append independently.
- When a predecessor buffer reaches 64 and the current buffer exceeds it, the two merge into one larger tree.

This yields $O(\log|R(s)|)$ blocks, $O(\log|R(s)|)$ tree rebuilds per vector, and amortized $O(\log|R(s)|)$ query time. For large residue sets (200+ vectors), this provides 10–20 $\times$ speedup over linear scans.

4 Correctness

Lemma 1 (Residue sufficiency). If cost $a \in C(s)$ is not in $F(s)$ and no frontier member truncation-dominates a , then a full dominator of a exists iff a residue member dominates a .

Proof. Follow the chain of evictors. Since $a \notin F(s)$, some expansion b_0 evicted it; either $b_0 \leq a$ (full dominator) or $b_{0,1} > a_{0,1}$. In the latter case, b_0 is in $R(s)$. If $b_0 \leq a$, we are done. Otherwise, trace successively earlier evictors: each is in $R(s)$ and forms a chain. The chain is finite and must terminate in $R(s)$ (by assumption, it cannot end in $F(s)$). ▀

Theorem 1 (Search equivalence). Under identical OPEN tie-breaking, successor order, and H ordering, FAST-MVH and standard search make the same OPEN insertions and return the same solutions.

Proof sketch. Both dominance tests are pure (no side effects). Their order is interchangeable. By Lemma 1, LocalDom is exact. Induction on extractions shows both searches maintain identical frontier coverage and make identical pruning decisions. ▀

5 Implementation and Optimizations

5.1 Adaptive k-d tree promotion

State frontiers $F(s)$ start as unindexed linear arrays. Indexing overhead (tree construction, pointer indirection) only pays off when frontiers are large and queries are frequent. FAST-MVH uses two criteria:

1. **Size threshold:** Frontier must contain ≥ 8 vectors.
2. **Cost threshold:** Average query cost (truncated comparisons per find) must exceed 64 comparisons.

Once promoted, a tree is maintained and rebuilt incrementally. Rebuild threshold: frontier size must exceed $1.25 \times (\text{size at last build}) + 64$. This amortization balances rebuild cost against progressive frontier growth.

Why this matters: Small frontiers (<8 vectors) are traversed linearly faster than via tree navigation. Sparse, tight frontiers benefit modestly from spatial pruning. By deferring tree construction, FAST-MVH avoids wasted overhead on transient states and small neighborhoods.

5.2 Witness caching and fallback strategies

When LocalDom scans $F(s)$ and finds only truncated dominators (witness $w = \text{TR}$), a fallback full-dimensional search is needed. Three strategies are possible:

1. **Re-scan $F(s)$ in full dimensions:** Simple, but redundant if many candidates share similar truncated dominators.
2. **Scan only $R(s)$:** Faster, but assumes no full dominator exists in $F(s)$ (Lemma 1 guarantees this).
3. **Cache the truncated witness and reuse it:** If a future query has the same truncated dominator, skip truncated re-scanning.

FAST-MVH uses strategy (2): fallback scans only $R(s)$. This is sound (Lemma 1) and fast (indexed). Additionally, if the same state s and similar costs g arrive repeatedly (common in search), witness caching can be extended to memoize the truncated-search result, avoiding even the indexed fallback. Current implementation focuses on fallback indexing; witness memoization is a secondary optimization.

5.3 Memory layout and cache locality

The implementation uses contiguous arrays and inline f values to exploit CPU cache hierarchies:

- **OPEN list:** Stores f values inline in heap entries, avoiding pointer chasing during comparisons.
- **State records:** Costs g and heuristic indices are packed in contiguous Rec structures, reducing cache misses during successor generation.
- **K-d tree nodes:** Medians-based tree layout (via radix permutation) preserves spatial locality within each tree block.

These decisions have no algorithmic impact but improve constant factors substantially on modern hardware.

5.4 Heuristic indexing (optional)

The set $H(s)$ can itself be indexed with a k-d tree over truncated coordinates, using bounding boxes around all heuristic vectors at s . If a bounding box’s minimum is dominated by the goal frontier, all heuristics in that box are refuted.

Current implementation disables this by default due to poor empirical pruning effectiveness; heuristic sets are typically small (5–20 vectors) and already reordered by redundancy pointers. Full-fledged bounding-box indexing shows promise on instances with 100+ heuristics per state but is not evaluated here.

6 Experiments

6.1 Methodology

Test problems span grid-based and real-world road networks across three to eight objectives. Grids are four-connected $n \times n$ graphs with integer edge costs; pairwise objective correlations range $\rho \in [-0.6, 0.0]$, creating anti-correlated and independent structures that stress frontier indexing. Road networks are DIMACS subgraphs (New York, Bay Area) at multiple scales, with synthetic multi-objective overlays.

Multi-valued heuristics are backward Pareto sets computed via A*pex, a state-of-the-art generator for multi-objective domains. All heuristic files are lexicographically sorted by their first coordinate—a correctness requirement for admissibility under dimensionality reduction.

Platform: Intel Core i5-1135G7, 16 GB RAM, Windows 11. **Build:** GCC 16.2, -O2 -DNDEBUG -std=c++20. Times exclude I/O and initialization.

6.2 Grid-based instances

Table 1: Grid instances spanning three to eight objectives.

Instance	M	Solutions	Expansions	Baseline (s)	FAST-MVH (s)	Speedup
Grid 10×10	3	1,602	9,500	0.058	0.032	1.8×
Grid 10×10 $\varepsilon=0.05$	4	11,330	53,652	1.738	0.338	5.1×
Grid 8×8 $\varepsilon=0.05$	5	9,412	33,005	1.473	0.241	6.1×
Grid 8×8 $\varepsilon=0.05$	6	70,196	179,297	92.031	2.336	39.4×
Grid n7 $\rho=-0.2$ $\varepsilon=0.05$	7	32,643	91,704	24.513	1.383	17.7×
Grid n8 $\rho=-0.2$ $\varepsilon=0.05$	8	47,079	99,244	36.956	1.435	25.8×

All solutions byte-identical to baseline.

Grid instances reveal systematic dimensional scaling: speedup grows from 1.8×

 at $M = 3$ to 39.4× at $M = 6$, then moderates at $M = 7$ and $M = 8$. The peak occurs where frontier variance is high (dense Pareto sets with anti-correlated objectives) and baseline

cost explodes due to quadratic full-dimensional comparisons.

At $M = 3$, truncated queries operate on only 2 coordinates, and frontier heights remain below tree-promotion thresholds throughout execution. Speedup primarily reflects inline- f storage and test-order optimization. From $M = 4$ onward, progressively larger frontiers meet promotion criteria; bounding-box pruning becomes effective. At $M = 6$, the baseline performs 81 million comparisons on 179K expansions; FAST-MVH performs 1.9 million, a $43\times$ reduction. The grid-8 instance at $M = 8$ shows moderate speedup ($25.8\times$) because frontier density increases but total expansion count is lower, reducing absolute fallback queries.

6.3 Road network instances

Table 2: DIMACS road networks at multiple dimensions and scales.

Instance	M	Solutions	Expansions	Baseline (s)	FAST-MVH (s)	Speedup
Bay Area 8	3	238	12,346	0.053	0.018	$2.9\times$
Bay Area 16	3	3,534	347,498	2.673	0.647	$4.1\times$
Bay Area 8 $\epsilon=0.1$	4	3,716	157,704	20.052	0.654	$30.7\times$
NY 5K $\epsilon=0.01$	4	7,105	144,117	9.551	1.039	$9.2\times$
NY 8K $\epsilon=0.05$	4	5,713	195,594	39.127	1.033	$37.9\times$
NY 5K $\epsilon=0.05$	5	26,701	503,505	183.722	5.459	$33.7\times$
NY 3K $\epsilon=0.05$	6	11,286	127,349	104.203	1.270	$82.0\times$
Bay Area 16 $\epsilon=0.1$	4	55,069	2,881,236	41.811	21.923	$1.9\times$
Bay Area 8 $\epsilon=0.1$	5	33,104	1,195,876	27.839	19.116	$1.5\times$

All solutions verified bit-identical.

Road networks exhibit wide variance ($1.5\times$ – $82\times$ speedup), determined by frontier structure and problem scale. Sparse network regions produce shallow, tight frontiers where truncated queries are already selective; bounding-box pruning has minimal additional effect. Bay Area 16 with 2.88M expansions at $M = 4$ achieves only $1.9\times$ speedup: most vectors remain in $F(s)$, fallback queries are rare, and residue forests provide little benefit.

Conversely, dense network regions and higher dimensions yield dramatic speedups. NY 3K at $M = 6$ (11K solutions, 127K expansions) achieves **$82\times$ speedup**: baseline performs 1,097 million full-dimensional comparisons; FAST-MVH performs 13.3 million—an 82-fold reduction matching overall speedup exactly. This instance exemplifies the algorithm’s sweet spot: large expansion count, dense frontier at goal, and sufficient dimensionality for bounding-box pruning to be highly selective.

6.4 Fallback and residue-forest efficiency

Table 3: Residue indexing contribution—full-dimensional comparison reduction.

Instance	M	Fallback Queries	Baseline Cmp (M)	FAST-MVH Cmp (M)	Reduction
Grid $M=4$	4	11,778	15.0	0.81	$18.5\times$
Grid $M=6$	6	14,804	81.2	1.86	$43.7\times$
NY 5K $M=4$	4	58,440	29.0	2.75	$10.5\times$
NY 5K $M=5$	5	91,934	93.2	4.83	$19.3\times$
NY 8K $M=4$	4	5,439	2.26	0.156	$14.5\times$
NY 3K $M=6$	6	333	1,097	13.3	$82.4\times$
Bay 8 $M=4$	4	6,414	0.707	0.051	$13.9\times$

Fallback queries = full-dimensional searches triggered by truncated-only dominators. Comparisons = millions of full-vector comparisons.

Residue sets retain 30–70% of all expanded vectors (those evicted with $p_1 < g_1$), yet full-dimensional searches are 10– $82\times$ faster. This gap arises entirely from logarithmic-method indexing: without k-d forest decomposition, fallback would be linear table scan.

The data reveals two regimes:

Low-dimensional and sparse instances: Fallback is rare because most vectors stay in $F(s)$ and are never evicted. Reduction factors are 10–20 $\times$; these are driven by effective bounding-box pruning when fallback does occur, not by the size of $R(s)$.

High-dimensional and dense instances: Fallback is frequent because frontier updates evict many vectors. Reduction factors exceed 40 $\times$; the logarithmic forest structure enables $O(\log|R(s)|)$ vs. $O(|R(s)|)$ linear scans. NY 3K $M=6$ is extreme: only 333 fallback queries among 127K expansions, yet each query traverses a residue of 1,700+ vectors. The forest reduces 1,097M potential comparisons to 13.3M.

6.5 Scaling analysis and key observations

Across all eighteen instances, FAST-MVH delivers consistent acceleration. Low-dimensional instances ($M \leq 3$) show modest speedup (1.8–2.9 $\times$), limited by small frontier heights and tree-promotion overhead. From $M = 4$ onward, speedup exceeds 5 $\times$, scaling nonlinearly to 82 $\times$ at $M = 6$ on favorable instances.

Speedup correlates most strongly with (i) frontier cardinality and variance, (ii) problem dimensionality, and (iii) Pareto frontier density at the goal. Anti-correlated objectives in grids produce dense frontiers and dramatic speedups; clustered objectives in road networks (many instances optimize related route qualities) yield sparser frontiers and modest gains.

FAST-MVH is not a universal win: instances with tight, small frontiers benefit modestly from indexing. However, across diverse real-world problem classes and dimensions $M \in [3, 8]$, the algorithm achieves multi-fold speedups with zero algorithmic error.

7 Conclusion

Spatial indexing of truncated frontiers and indexed residue sets reduce the cost of dominance checking in multi-objective search with multi-valued heuristics. Four synergistic design choices—adaptive k-d trees on truncated frontiers, local-first test order, logarithmically decomposed residue indexing, and witness caching—achieve 1.8 $\times$ to 82 $\times$ speedups without changing search decisions, expansion sequences, or solution sets. These speedups come at modest memory cost and make FAST-MVH practical for high-dimensional problems where dominance checking has historically dominated search time.

References

1. Bentley, J. L. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Communications of the ACM* 18(9): 509–517.
2. Bentley, J. L.; and Saxe, J. B. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1(4): 301–358.
3. Demetrescu, C.; Goldberg, A. V.; and Johnson, D. S., eds. 2009. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*. DIMACS Series 74. American Mathematical Society.
4. Zhang, H.; Salzman, O.; Kumar, T. K. S.; Hernandez Ulloa, C.; Suazo, L.; and Koenig, S. 2022. A*pex: Efficient Approximate Multi-Objective Search on Graphs. *Proceedings of ICAPS*, 394–403.